

ECE 468: Lab 2 Report

UNIVERSITY OF
Waterloo



Department of Electrical and Computer Engineering

A Report Prepared For:

The University of Waterloo
Dr. Gennaro Notomista
ECE468

Prepared By:

Hanhua Ge
Ameen Aydan
150 University Ave W.
Waterloo, Ontario, N2N 2N2

June 17th, 2026

Table of Contents

The Derivation	3
Joint Angle Relationships.....	3
Forward Kinematics	4
Offline Validation	4
Workspace	5
Workspace Exploration	5
Workspace Function	6
Validation.....	7
Camera Matrix Acquisition	8

The Derivation

Joint Angle Relationships

Figure 1 below shows a diagram depicting the joint angle relationship.

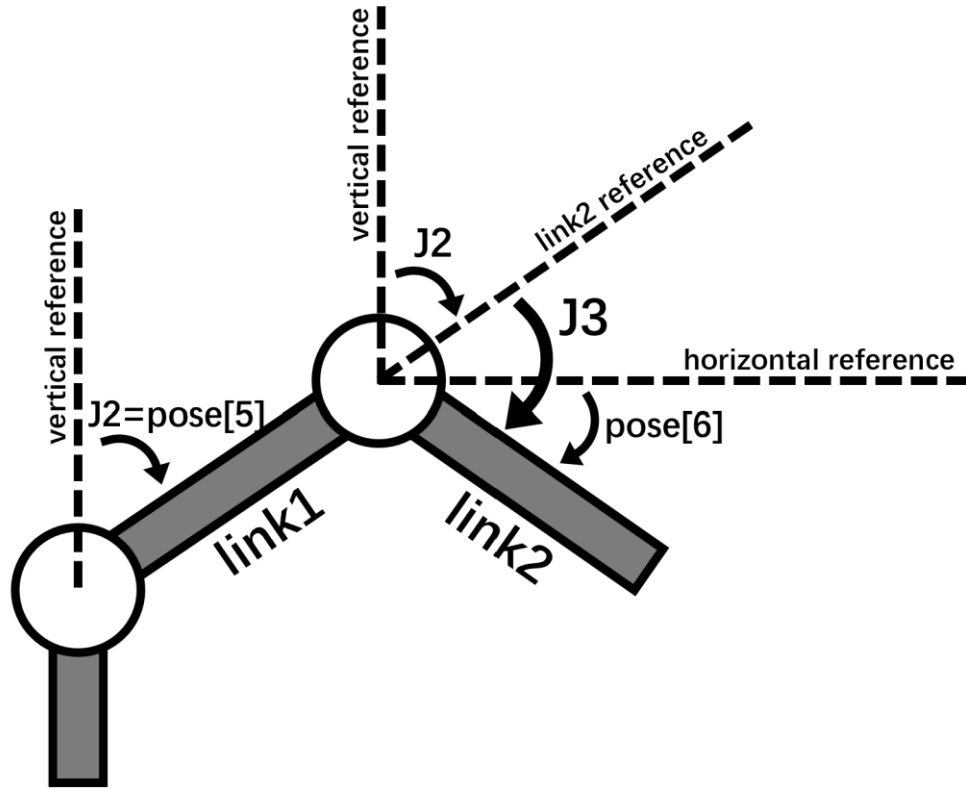


Figure 1: Diagram of joint angle relationship

The definition of the data set dictionary is following:

$\text{pose}[5]$ = angle of link 1 from vertical

$\text{pose}[6]$ = angle of link 2 from horizontal

Let J_2, J_3 be the two joint angle respectively, according to the graph we draw, we can obviously see that the relationships of poses towards the joint angle are

$$J_2 = \text{pose}[5]$$

$$J_3 = \text{pose}[6] + \frac{\pi}{2} - J_2$$

Forward Kinematics

In our code, we directly use $pose[4]$, $pose[5]$, $pose[6]$, or equivalently J_1 , J_2 , and FA (forearm angle) to do the forward kinematic calculations. Let l_1 and l_2 be the length of two links, and EE_o define as the offset of end effector (EE), which is a short link that always horizontal to the ground. This is shown in figure 2.

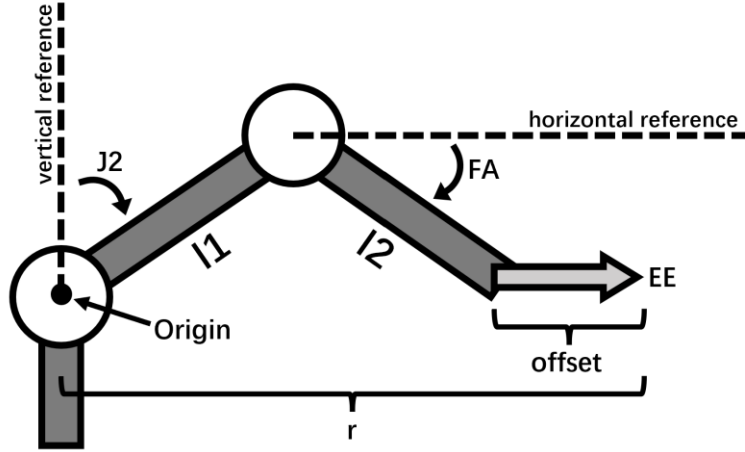


Figure 2: Defining the robot for forward kinematics

Basically, we set the origin at the joint shown on picture, and we first calculate z coordinate of EE and the radius r , which defined as the horizontal distance the EE can reach away from origin on reference of xy -plane.

$$z = l_1 \cos(J_2) - l_2 \sin(FA)$$

$$r = l_1 \sin(J_2) + l_2 \cos(FA) + EE_o$$

After we calculate the radius, we can easily calculate the x and y coordinate as

$$x = r \cos(J_1)$$

$$y = r \sin(J_1)$$

Offline Validation

The txt file provided contains lists of joints angles (J_1, J_2, FA) with corresponding cartesian coordinates (x, y, z). For validation, we use the forward kinematic function we wrote to convert the joint angle coordinates to cartesian coordinates (x_{FK}, y_{FK}, z_{FK}), and calculate the error by Euclidean distance

$$e = \|(x - x_{FK}, y - y_{FK}, z - z_{FK})\| = \sqrt{(x - x_{FK})^2 + (y - y_{FK})^2 + (z - z_{FK})^2}$$

According to the validation, the error is basically under 10^{-5} mm level.

Mean error: 0.00001411 mm

Max error: 0.00005277 mm

Min error: 0.00000056 mm

Therefore, the derived forward kinematics model accurately reproduces the provided validation dataset.

Workspace

Workspace Exploration

We use several different boundary points for test the workspace exploration; the code successfully gives us the joint angles of each boundary points. This is shown in table 1 below.

Table 1: Workspace exploration targets

Boundary Type	Cartesian Coordinate (mm)	J1 (deg)	J2 (deg)	FA (deg)
Inner Boundary	(140, 0, -10)	0.00	15.43	72.42
Outer Boundary	(260, 0, -10)	0.00	45.72	45.17
Upper Boundary	(200, 0, 0)	0.00	25.49	56.00
Lower Boundary	(200, 0, -57)	0.00	50.67	75.88
Left Boundary	(10, 230, -10)	87.51	37.36	52.94

The inner boundary corresponds to a folded arm configuration, resulting in a relatively small J_2 and large forearm angle. The outer boundary requires the arm to extend outward, increasing J_2 while reducing forearm angle. The left boundary requires a large J_1 to rotate the base to corresponding direction. The upper boundary corresponds to the highest tested end-effector position, which required a moderate J_2 angle and forearm angle to keep the arm elevated while staying within the radial workspace. The lower boundary required significantly larger J_2 and forearm angles, indicating that the manipulator had to rotate further downward to reach a lower elevation.

Note: The desired lower boundary points should have $z = -120$, and the desired left boundary should have $x = 0$. However, when we test these points in the simulator, we noticed that these points will be marked as reachable points, and the simulated manipulator will try to move to that point, but they are actually unable to reach due to some internal settings or limits of the simulator. Therefore, in real lab, we keep the coordinate same as simulator in order to prevent the possible error or broken of the real manipulator.

Workspace Function

To determine whether a joint configuration lies within the workspace, we basically first convert it into cartesian coordinate, and then call and use the previous workspace function in Lab1, which basically checks whether following conditions are all satisfied

$$-120 \leq z \leq 0$$

$$140 \leq \sqrt{x^2 + y^2} \leq 260$$

$$x \geq 0$$

Points located exactly on the workspace boundary were treated as valid, inclusive inequalities were used in the implementation. The code is shown in figure 3 and 4 below.

```
def is_joint_in_workspace(J1, J2, FA):  
    x, y, z = forward_kinematics(J1, J2, FA)  
    return is_point_in_workspace(x, y, z)
```

Figure 3: Joint in workspace validation function

```
def is_point_in_workspace(x, y, z):  
  
    r = np.sqrt(x**2 + y**2)  
  
    return (  
        -120 <= z <= 0 and  
        140 <= r <= 260 and  
        x >= 0  
    )
```

Figure 4: Point in workspace validation function

All joint configurations used during the real robot validation experiments were first evaluated using the workspace validation function. The function successfully classified all tested configurations as valid, and each configuration was executed without violating the workspace constraints. These results provide confidence that the workspace validation function correctly identifies reachable robot configurations. The results are shown in table 2 below.

Table 2: Joint angle validation results

Joint Angle	Workspace Validation
(0,20,70)	Valid
(45,40,70)	Valid
(-45,20,70)	Valid
(0,30,60)	Valid

(20,30,60)	Valid
(-20,30,60)	Valid

Validation

The forward kinematics model was validated by comparing the cartesian end-effector position that is predicted with the measured pose data. For each point, the measured joint angles J_1, J_2 and, J_3 were substituted into the forward kinematics (FK) equation to get the estimated position. The predicted position was then compared to the measured cartesian position using Euclidean position error:

$$e = \sqrt{(x_{\{FK\}} - x_{\{meas\}})^2 + (y_{\{FK\}} - y_{\{meas\}})^2 + (z_{\{FK\}} - z_{\{meas\}})^2}$$

For each validation test, the desired joint angles are first checked with the workspace validation function. If the configuration was valid, the robot was commanded with the joint-angle motion. The measured pose data of the physical robot is then substituted into the FK equation and the resulting cartesian position is compared with the measured position from the same pose output.

The points were selected to test multiple regions of the *usable* workspace. The first and third tests use a small radial reach with $J_1 = 0^\circ$ and $J_2 = -45^\circ$, which check the model near the inner radial region and in the negative y-direction. The second test uses $J_1 = 45^\circ$ with a larger J_2 which checks a diagonal configuration of the robot. The final three tests use the same J_2 and forearm angle while varying J_1 between 0, 20 and -20 degrees. This isolates the effect of base rotation and verifies that the radial distance is being correctly projected in the x and y directions. The results of the validation procedure are shown in table 3 below.

Table 3: Validation results

Commanded joints in degrees (J_1, J_2, FA)	Actual joints in degrees (J_1, J_2, FA)	FK position (mm)	Measured position (mm)	Error (mm)
[0, 20, 70]	[0.00, 20.00, 70.00]	[156.15, 0.00, -11.28]	[156.1497, 0.0000, -11.2763]	0.0000121499
[45, 40, 70]	[45.00, 40.00, 70.00]	[139.13, 139.13, -34.72]	[139.1255, 139.1256, -34.7188]	0.0000404699
[-45, 20, 70]	[-45.00, 20.00, 70.00]	[110.41, -110.41, -11.28]	[110.4145, -110.4145, -11.2763]	0.0000203717
[0, 30, 60]	[0.00, 30.00, 60.00]	[200.70, 0.00, -10.39]	[200.7000, 0.0000, -10.3923]	0.0000183624

[20, 30, 60]	[20.00, 30.00, 60.00]	[188.60, 68.64, -10.39]	[188.5963, 68.6434 - 10.3923]	0.0000111500
[-20, 30, 60]	[-20.00, 30.00, 60.00]	[188.60, -68.64, -10.39]	[188.5963, -68.6434, -10.3923]	0.0000111500

The numerical error is summarized by using mean error, maximum error, minimum error and RMS error for all 10 runs of the validation scheme. This is shown below:

$$\overline{(e)} = \frac{1}{n} \sum_{i=1}^n e_i$$

$$e_{RMS} = \sqrt{\frac{1}{n} \sum_{i=1}^n e_i^2}$$

The above equations are used to find the error statistics below:

$$\overline{(e)} = 1.8942 \times 10^{-5} mm$$

$$e_{max} = 4.0470 \times 10^{-5} mm$$

$$e_{min} = 1.1150 \times 10^{-5} mm$$

$$e_{RMS} = 2.1549 \times 10^{-5} mm$$

The maximum error across all runs was still less than $10^{-4} mm$, which is far below the physical accuracy expected from the robot. This indicates that the derived FK model is consistent with the pose convention and prediction of the cartesian end effector position from the measured joint angles.

The physical validation results are also consistent with the offline validation numbers, where there is a mean error of $1.411 \times 10^{-5} mm$ and a maximum error of $5.277 \times 10^{-5} mm$. When compared to the error of the real robot, they are of the same order of magnitude. This means the same FK model behaves consistently across both data sets. The remaining discrepancy is likely rounding error in the FK values, floating point precision and the limited number of displayed decimal places.

Camera Matrix Acquisition

The camera matrix was obtained using the provided python script. The lab procedures from the manual were followed to obtain the intrinsic camera parameters for the camera mounted at the lab station. The general camera matrix is shown as:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

To perform calibration, the camera was connected to the computer and chessboard calibration target was placed within camera view. The python script was run, and then 10 images were captured while moving the chessboard to different position by rotating and translating. The image was only captured when the detected corner overlay was visible, which confirms that the chessboard had been identified correctly.

The three camera matrices obtained from 3 separate runs are shown below. These were taken three different times to confirm that the values remained within an acceptable error margin. The three calibration runs produced camera matrices with the same overall structure but slightly different numerical values. This is expected because the camera matrix is estimated from real image data, and the result depends on the exact chessboard poses, lighting conditions, detected corner locations, and numerical optimization.

$$K_1 = \begin{bmatrix} 819.11799606 & 0 & 312.66588670 \\ 0 & 860.76055989 & 250.12833571 \\ 0 & 0 & 1 \end{bmatrix}$$

$$K_2 = \begin{bmatrix} 735.15034660 & 0 & 321.02515241 \\ 0 & 753.36734873 & 244.42060448 \\ 0 & 0 & 1 \end{bmatrix}$$

$$K_3 = \begin{bmatrix} 767.58943884 & 0 & 311.18207926 \\ 0 & 801.12060803 & 260.56396707 \\ 0 & 0 & 1 \end{bmatrix}$$